

CBeave Version 1

Entity-Relationship Diagram Guide

Data model summary, field reference, use cases, and end-to-end workflows

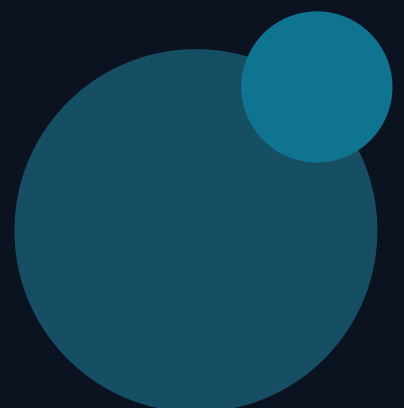
Implementation baseline: SRS v1.1

Database: PostgreSQL with Prisma ORM

Prepared: 2026-07-14

Scope note

This guide documents the approved one-month V1 model. Payments, shipping, password recovery, external email delivery, messaging, reviews, storefronts, and advanced analytics remain outside V1.



How to use this guide

The ERD is organized into four domains. Each table section explains why the table exists, how it relates to other tables, what every field means, a realistic use case, and the expected service workflow. The final section combines tables into complete product journeys.

Domain	Tables	Responsibility
Identity and Access	users, user_profiles, auth_accounts, user_sessions	Registration, login, Google linking, roles, status, and refresh sessions.
Auction Core	categories, auctions, auction_images, bids, auction_extensions, auction_events	Taxonomy, lifecycle, pricing, real-time bid state, anti-sniping, and history.
Engagement	watchlists, auction_participants, notifications	Saved auctions, Live Arena participation, and in-app alerts.
Governance	auction_reports, admin_actions	Trust-and-safety review and immutable administrator audit.

Key modeling principle

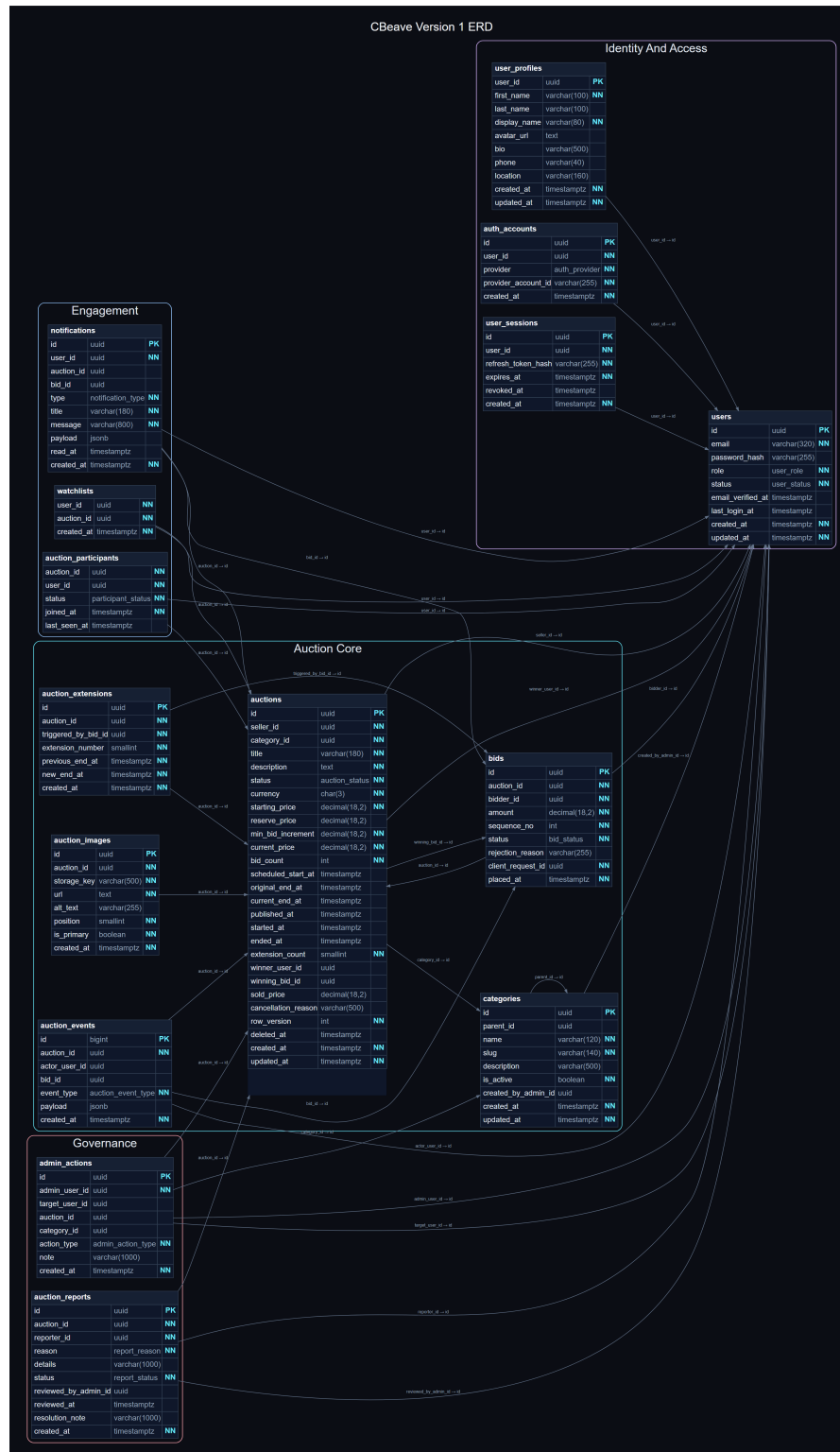
The auctions row is the authoritative current aggregate. bids, auction_extensions, auction_events, notifications, and admin_actions preserve ordered facts and side effects around that state.

Core rules

- Guest is not a stored role. Persisted roles are USER and ADMIN.
 - One USER may sell and bid, but never bid on an auction they own.
 - An ADMIN moderates but cannot sell, watch as a buyer, or bid.
 - Preview is UI-only and Publish is a command; neither is an auction status.
 - Reserve price is private. Only reserveMet may be buyer-facing.
 - Accepted bid creation and all auction counter/extension changes are atomic and idempotent.
 - Referenced categories are deactivated rather than destructively deleted.

Relationship overview

The full ERD below is an orientation map. Exact fields and workflows are documented in the following sections. The Version 1 auction aggregate now derives reserve state and uses accepted bid count for simple Hot Auctions; curated Featured Auctions are deferred.



Enum reference

Enums constrain lifecycle and action values so services and clients do not invent inconsistent strings. Transitions must still be validated in application services.

Enum	Values	Use
user_role	USER, ADMIN	Separates marketplace participation from platform administration.
user_status	ACTIVE, SUSPENDED, DEACTIVATED	Controls login and all protected operations.
auth_provider	GOOGLE	External identity providers supported in V1.
auction_status	DRAFT, SCHEDULED, ACTIVE, SOLD, UNSOLD, CANCELLED	Canonical persisted lifecycle.
bid_status	ACCEPTED, REJECTED, RETRACTED	Bid outcome; RETRACTED requires a defined admin workflow.
participant_status	RSVP, JOINED, LEFT	Persistent Live Arena participation state.
notification_type	OUTBID, AUCTION_WON, AUCTION_ENDED, AUCTION_CANCELLED	In-app notification categories.
report_status	OPEN, REVIEWING, RESOLVED, DISMISSED	Moderation report workflow.
report_reason	PROHIBITED_ITEM, MISLEADING_CONTENT, COPYRIGHT, FRAUD, OTHER	User-selected trust-and-safety category.
auction_event_type	CREATED, PUBLISHED, STARTED, BID_PLACED, EXTENDED, ENDED, CANCELLED	Append-only auction history.
admin_action_type	SUSPEND_USER, REACTIVATE_USER, CANCEL_AUCTION, CREATE_CATEGORY, UPDATE_CATEGORY, DEACTIVATE_CATEGORY, RESOLVE_REPORT	Audited V1 administrator operations.

Lifecycle guard

Database enums restrict values, but they do not prevent an invalid transition such as SOLD back to ACTIVE. Services must enforce the transition matrix.

Identity and Access

4 tables

Domain responsibility

Account identity, public profile, Google linking, authorization status, and revocable refresh sessions.

IDENTITY AND ACCESS

users

Purpose

Stores the account identity, authorization role, account status, and core authentication state for every standard user and administrator.

Related requirements: AUTH-001, AUTH-002, AUTH-003, USR-001, ADM-002

Relationships

- One user has one user_profiles row.
- One user can have many auth_accounts and user_sessions.
- A USER can own auctions, place bids, watch auctions, join Live Arena, receive notifications, and submit reports.
- An ADMIN can create admin_actions and review auction_reports, but cannot sell or bid.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Stable account identifier referenced throughout the schema.
email	varchar(320)	Required, unique	Login address and account identity. Normalize consistently before uniqueness checks.
password_hash	varchar(255)	Nullable	Password hash for local accounts; null is permitted for Google-only accounts.
role	user_role	Required, USER default	Authorization role: USER or ADMIN.
status	user_status	Required, ACTIVE default	ACTIVE, SUSPENDED, or DEACTIVATED account state.
email_verified_at	timestampz	Optional	When local email ownership was verified, if verification is implemented.
last_login_at	timestampz	Optional	Most recent successful login time.
created_at	timestampz	Required	Account creation time.
updated_at	timestampz	Required	Most recent account record update.

Use case

User A registers locally. The API inserts an ACTIVE USER row with a password hash. An administrator later suspends the account, revokes its sessions, and records SUSPEND_USER in admin_actions.

Table workflow

1	Create identity Validate and normalize email, hash the password for a local account, and insert the user.
2	Authorize Server guards check role and status. UI checks alone are never sufficient.
3	Use marketplace An active USER may sell and buy, but cannot bid on an auction they own.
4	Moderate Suspension changes status, revokes active sessions, and creates an audit action in one transaction.

Integrity and implementation notes

- Never return password_hash in an API response.
- A suspended or deactivated account must fail login, refresh, auction creation, and bidding checks.
- The (role, status) index supports admin account filtering and authorization operations.

IDENTITY AND ACCESS

user_profiles

Purpose

Stores editable personal and public profile information separately from authentication and authorization fields.

Related requirements: AUTH-001, USR-001

Relationships

- Exactly one profile belongs to one users row through user_id.
 - user_id is both the primary key and foreign key, enforcing a one-to-one relationship.

Field reference

Field	Type	Rules	Meaning
user_id	uuid	PK, FK users.id	Owner of the profile.
first_name	varchar(100)	Required	Given or preferred first name.
last_name	varchar(100)	Optional	Optional family name, supporting single-name users.
display_name	varchar(80)	Required	Public identity shown in auctions and masked bidder displays.
avatar_url	text	Optional	Profile image location.
bio	varchar(500)	Optional	Short user biography.
phone	varchar(40)	Optional	Private contact information; not public auction data.
location	varchar(160)	Optional	User-entered location text.
created_at	timestamptz	Required	Profile creation time.
updated_at	timestamptz	Required	Profile modification time.

Use case

A user named Cher updates display_name to CBeaverFan. Auction pages use the display name, while the application derives a full profile name from first_name plus optional last_name.

Table workflow

1	Register Create users and user_profiles atomically so every marketplace account has a usable public identity.
2	Update The owner changes allowed fields through /users/me after validation.
3	Present Buyer-facing views use display_name or a masked form, not private phone or location data.

Integrity and implementation notes

- Do not persist a separate full_name value; derive it to avoid inconsistent duplicate data.
 - display_name is indexed for profile lookup but is not currently declared unique.
 - Treat phone and location as private fields in DTOs and authorization policies.

IDENTITY AND ACCESS

auth_accounts

Purpose

Links an external Google identity to a CBeave user without mixing provider identifiers into the users table.

Related requirements: AUTH-003

Relationships

- Many provider accounts can conceptually belong to one user, although V1 supports only GOOGLE.
- Each Google provider account identifier can link to only one CBeave user.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Internal link identifier.
user_id	uuid	Required, FK users.id	CBeave account owning the provider link.
provider	auth_provider	Required	External provider; GOOGLE in V1.
provider_account_id	varchar(255)	Required	Stable account identifier returned by Google.
created_at	timestampz	Required	When the provider link was created.

Use case

Google returns provider account 1042. The callback finds the unique GOOGLE plus 1042 link and signs in its user. If no link exists, it creates users, user_profiles, and auth_accounts in one transaction.

Table workflow

1	Verify Google Validate the OAuth response and obtain the stable provider account identifier.
2	Find link Query by provider and provider_account_id.
3	Create or sign in Create a new user/profile/link when allowed, otherwise load the existing user.
4	Create session Issue the same user_sessions refresh flow used by local login.

Integrity and implementation notes

- Unique(provider, provider_account_id) prevents one Google identity from mapping to multiple users.
- Unique(user_id, provider) prevents duplicate Google links for the same user in V1.
- Do not trust email alone as the provider identifier.

IDENTITY AND ACCESS

user_sessions

Purpose

Represents revocable refresh-token sessions so short-lived access tokens can be renewed and logout can invalidate a device session.

Related requirements: AUTH-002, AUTH-004

Relationships

- One user can have many sessions, such as one per browser or device.
- The session belongs to users through user_id.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Internal session identifier.
user_id	uuid	Required, FK users.id	Account owning the session.
refresh_token_hash	varchar(255)	Required, unique	Hash of the secret refresh token. The raw token is never stored.
expires_at	timestampz	Required	Hard expiry after which refresh is rejected.
revoked_at	timestampz	Optional	Logout or security revocation time; null means not explicitly revoked.
created_at	timestampz	Required	Session creation time.

Use case

User B logs in on a laptop. The API stores a refresh-token hash and sends the raw token in a secure HttpOnly cookie. Logout sets revoked_at and clears the cookie.

Table workflow

1	Login Generate a short-lived access token and a high-entropy refresh token.
2	Persist Hash the refresh token and store the hash, owner, expiry, and creation time.
3	Refresh Hash the presented cookie, find the session, and require no revocation, future expiry, and ACTIVE user status.
4	Rotate Issue a new refresh token and replace the stored hash when the session is refreshed.
5	Logout Set revoked_at and clear the client cookie. Suspension revokes all sessions for the user.

Integrity and implementation notes

- V1 intentionally excludes IP-address, geolocation, and user-agent metadata.
- The (user_id, expires_at) index supports listing and revoking a user's sessions.
- Use HttpOnly, Secure in production, and an appropriate SameSite cookie policy.

Auction Core

6 tables

Domain responsibility

The authoritative auction aggregate and the ordered records that support images, bids, anti-sniping, and lifecycle history.

AUCTION CORE

categories

Purpose

Provides the administrator-managed taxonomy used to organize and filter auctions.

Related requirements: CAT-001, ADM-003, ADM-004

Relationships

- A category can have an optional parent category through `parent_id`.
- One category can classify many auctions.
- `created_by_admin_id` optionally records the administrator who created it.

Field reference

Field	Type	Rules	Meaning
<code>id</code>	uuid	PK, generated	Category identifier.
<code>parent_id</code>	uuid	Optional, self FK	Parent category for a simple hierarchy.
<code>name</code>	varchar(120)	Required	Human-readable category name.
<code>slug</code>	varchar(140)	Required, unique	Stable URL and query identifier.
<code>description</code>	varchar(500)	Optional	Administrator-managed category explanation.
<code>is_active</code>	boolean	Required, true default	Whether sellers may select the category for new/published auctions.
<code>created_by_admin_id</code>	uuid	Optional, FK users.id	Administrator responsible for creation.
<code>created_at</code>	timestampz	Required	Creation time.
<code>updated_at</code>	timestampz	Required	Last modification time.

Use case

An administrator deactivates Electronics after replacing it with Consumer Electronics. Existing auctions keep their `category_id`, but new auctions cannot select the inactive category.

Table workflow

1	Create Admin validates name, unique slug, and optional parent; insert the category and <code>CREATE_CATEGORY</code> audit action.
2	Use Public category selectors query only <code>is_active = true</code> .
3	Update Admin edits metadata and records <code>UPDATE_CATEGORY</code> .
4	Deactivate Set <code>is_active = false</code> and record <code>DEACTIVATE_CATEGORY</code> in the same transaction.
5	Reactivate Set <code>is_active = true</code> . The current V1 enum may record this as <code>UPDATE_CATEGORY</code> unless <code>ACTIVATE_CATEGORY</code> is added later.

Integrity and implementation notes

- Referenced categories are deactivated rather than deleted.
- Drafts assigned to an inactive category should be blocked from publication until changed.
- Prevent parent cycles and self-parenting in the service layer.

AUCTION CORE

auctions

Purpose

Central aggregate for ownership, lifecycle, pricing, schedule, real-time counters, simple Hot Auctions ranking, winner, and moderation state.

Related requirements: AUC-001 through AUC-008, BID-001 through BID-004, ADM-001

Relationships

- Each auction belongs to one seller and one category.
- An auction has many images, bids, extensions, events, watchers, participants, notifications, and reports.
- winner_user_id and winning_bid_id are set only for SOLD auctions.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Auction identifier.
seller_id	uuid	Required, FK users.id	USER account that owns the auction.
category_id	uuid	Required, FK categories.id	Classification used for discovery.
title	varchar(180)	Required	Buyer-facing title.
description	text	Required	Buyer-facing item description.
status	auction_status	Required, DRAFT default	DRAFT, SCHEDULED, ACTIVE, SOLD, UNSOLD, or CANCELLED.
currency	char(3)	Required, USD default	ISO-style currency code for monetary fields.
starting_price	decimal(18,2)	Required	Initial auction price; greater than zero.
reserve_price	decimal(18,2)	Private, optional	Hidden sale threshold; reserveMet is derived, not persisted.
min_bid_increment	decimal(18,2)	Required	Minimum increase above current price.
current_price	decimal(18,2)	Required	Live accepted-highest-bid aggregate.
bid_count	int	Required, zero default	Accepted bid count and simple Hot Auctions ranking input.
scheduled_start_at	timestampz	Optional until publish	Scheduled activation time.
original_end_at	timestampz	Optional until publish	End time before anti-sniping extensions.
current_end_at	timestampz	Optional until publish	Effective deadline and Hot Auctions tie-breaker.
published_at	timestampz	Optional	When the draft was published.
started_at	timestampz	Optional	Actual activation time.
ended_at	timestampz	Optional	Actual terminal completion time.
extension_count	smallint	Required, zero default	Anti-sniping extension count; maximum five.
winner_user_id	uuid	Optional, FK users.id	Winner for a SOLD auction.
winning_bid_id	uuid	Optional, FK bids.id	Accepted winning bid for a SOLD auction.
sold_price	decimal(18,2)	Optional	Final accepted sale price.
cancellation_reason	varchar(500)	Optional	Seller/admin/system cancellation explanation.
row_version	int	Required, one default	Optimistic concurrency token.
deleted_at	timestampz	Optional	Soft-deletion marker; lifecycle status remains authoritative.
created_at	timestampz	Required	Draft creation time.
updated_at	timestampz	Required	Last aggregate update.

Use case

Seller A publishes a scheduled camera auction. At start time it becomes ACTIVE. Competing bids update current_price and bid_count. A final-two-minute bid extends current_end_at. At end, the highest reserve-meeting bid sets SOLD, winner_user_id, winning_bid_id, and sold_price.

Table workflow

1	Draft Create private editable auction data, images, pricing, and schedule.
2	Validate and publish Require active category, positive pricing, valid times, and at least one image. Set SCHEDULED or ACTIVE.
3	Activate Scheduler sets ACTIVE and started_at, then emits STARTED event and WebSocket update.
4	Bid Transaction validates bidder and amount, inserts bid, updates counters and reserve state, and may extend the end time.
5	Complete Scheduler decides SOLD or UNSOLD and persists terminal data and notifications.
6	Cancel Only DRAFT/SCHEDULED seller cancellation or explicit admin moderation is allowed; record reason and event.

Integrity and implementation notes

- Preview is not a status; it is a frontend rendering step.
- Publish is a command, not a status.
- Reserve amount is private; buyer APIs expose only reserveMet.
- current_price, bid_count, reserve evaluation, bid creation, and extension must be atomic.
- Status and schedule indexes support discovery and lifecycle workers.

AUCTION CORE

auction_images

Purpose

Stores ordered image metadata for each auction while the actual binary image lives in object/file storage.

Related requirements: AUC-001, AUC-002

Relationships

- Many image records belong to one auction through auction_id.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Image metadata identifier.
auction_id	uuid	Required, FK auctions.id	Owning auction.
storage_key	varchar(500)	Required, unique	Stable storage object key used for deletion or replacement.
url	text	Required	Delivery URL or resolved asset location.
alt_text	varchar(255)	Optional	Accessible description.
position	smallint	Required, zero default	Display ordering within the auction gallery.
is_primary	boolean	Required, false default	Main listing thumbnail flag.
created_at	timestampz	Required	Metadata creation time.

Use case

Seller uploads three images. Metadata positions 0, 1, and 2 define gallery order, while the position-0 image is marked primary for cards and search results.

Table workflow

1	Upload Validate file type/size, store the binary, then create metadata.
2	Order Assign unique auction_id plus position values.
3	Select primary Ensure one image is primary in service logic.
4	Publish check Reject publication when the auction has no image.

Integrity and implementation notes

- Unique(auction_id, position) prevents duplicate gallery positions.
 - The schema does not itself enforce exactly one primary image; use a transaction or partial unique index.
 - Delete storage objects only after authorization and consistent metadata changes.

AUCTION CORE

bids

Purpose

Stores ordered bid attempts and the idempotency key used to prevent duplicated bids caused by retries.

Related requirements: BID-001 through BID-005

Relationships

- Each bid belongs to one auction and one bidder.
 - One bid may trigger one auction_extensions record.
 - auctions.winning_bid_id may point to the final accepted bid.
 - Notifications and auction_events may reference a bid.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Bid identifier.
auction_id	uuid	Required, FK auctions.id	Auction receiving the bid.
bidder_id	uuid	Required, FK users.id	USER submitting the bid.
amount	decimal(18,2)	Required	Offered amount in the auction currency.
sequence_no	int	Required	Strict order within an auction.
status	bid_status	Required, ACCEPTED default	ACCEPTED, REJECTED, or RETRACTED.
rejection_reason	varchar(255)	Optional	Reason for a persisted rejected attempt, if rejected attempts are stored.
client_request_id	uuid	Required, unique	Client-generated idempotency key.
placed_at	timestampz	Required	Bid submission/acceptance time.

Use case

Bidder B submits 120 USD with request key K. A retry sends K again. The unique key returns the original result rather than creating a second bid.

Table workflow

1	Receive Validate input and locate any existing client_request_id.
2	Lock/serialize Protect the auction row or use equivalent concurrency control.
3	Validate Require ACTIVE auction, active USER, non-seller bidder, sufficient amount, and current time before current_end_at.
4	Accept Insert the bid and atomically update current_price, bid_count, sequence, reserve state, and possible extension.
5	Notify and broadcast Create outbid notification/event and publish the accepted state after commit.

Integrity and implementation notes

- Unique(auction_id, sequence_no) provides deterministic bid order.
 - Normal users should not retract bids. RETRACTED requires a carefully defined administrative recalculation workflow.
 - Rejected attempts may instead be security logs; the SRS strictly requires accepted-bid persistence.

AUCTION CORE

auction_extensions

Purpose

Persists every anti-sniping extension and the accepted bid that triggered it.

Related requirements: BID-004, LIV-003

Relationships

- Many extensions can belong to one auction, with a maximum of five.
- Each extension references exactly one triggering bid.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Extension identifier.
auction_id	uuid	Required, FK auctions.id	Extended auction.
triggered_by_bid_id	uuid	Required, unique FK bids.id	Accepted bid that caused the extension.
extension_number	smallint	Required	Sequential extension number 1 through 5.
previous_end_at	timestampz	Required	Effective end time before extension.
new_end_at	timestampz	Required	Effective end time after adding two minutes.
created_at	timestampz	Required	Extension creation time.

Use case

With 45 seconds remaining, an accepted bid arrives. Extension 2 records the old end and new end, while auctions.current_end_at and extension_count are updated atomically.

Table workflow

1	Check window After bid validation, determine whether current_end_at minus now is at most two minutes.
2	Check cap Require extension_count below five.
3	Persist Insert extension history and update auction end time/count in the same bid transaction.
4	Publish Create EXTENDED event and broadcast the new end time and extension number.

Integrity and implementation notes

- Unique(auction_id, extension_number) prevents duplicate sequence numbers.
- previous_end_at and new_end_at make extension history independently auditable.
- Never calculate the new deadline only in the browser.

AUCTION CORE

[auction_events](#)

Purpose

Append-only domain history for important auction lifecycle and bidding events, supporting audit, debugging, reconnect context, and event-driven updates.

Related requirements: AUC-004, AUC-007, BID-003, BID-004, ADM-001

Relationships

- Many events belong to one auction.
 - actor_user_id identifies a user/admin actor when applicable.
 - bid_id links bid-driven events to the accepted bid.

Field reference

Field	Type	Rules	Meaning
id	bigint	PK, increment	Monotonic event identifier useful for ordered replay.
auction_id	uuid	Required, FK auctions.id	Auction whose state changed.
actor_user_id	uuid	Optional, FK users.id	Seller, bidder, or administrator causing the event; null for system events.
bid_id	uuid	Optional, FK bids.id	Associated accepted bid.
event_type	auction_event_type	Required	CREATED, PUBLISHED, STARTED, BID_PLACED, EXTENDED, ENDED, or CANCELLED.
payload	jsonb	Optional	Event-specific snapshot such as price, count, or end time.
created_at	timestampz	Required	Event occurrence time.

Use case

A reconnecting Live Arena client supplies its last event id. The API can query later events for that auction and then refresh the authoritative auction state.

Table workflow

1	State transition A service updates the authoritative aggregate in a transaction.
2	Append event Insert the corresponding event in the same transaction whenever practical.
3	Deliver After commit, broadcast a WebSocket contract derived from committed data.
4	Inspect Operators query ordered events to understand lifecycle or bidding incidents.

Integrity and implementation notes

- Treat rows as append-only; corrections should create new events rather than rewriting history.
 - The events table is not the source of current auction state; auctions is authoritative in V1.
 - Avoid placing secrets, reserve amount, or private bidder data in payload.

Engagement

3 tables

Domain responsibility

User relationships around auctions: saved listings, Live Arena participation, and in-app notifications.

ENGAGEMENT

watchlists

Purpose

Stores a user's saved auctions as a simple many-to-many relationship.

Related requirements: WAT-001, WAT-002

Relationships

- A user can watch many auctions; an auction can be watched by many users.

Field reference

Field	Type	Rules	Meaning
user_id	uuid	Composite PK, FK users.id	User saving the auction.
auction_id	uuid	Composite PK, FK auctions.id	Saved public auction.
created_at	timestampz	Required	When the watch was added.

Use case

User B watches a scheduled auction. The watchlist page joins auctions to show status, current price, countdown, and eventual result.

Table workflow

1	Add Authenticate active USER, confirm public auction, and insert the composite key.
2	Idempotency If the composite key already exists, return the existing watched state.
3	List Query by user_id and join the current auction summary.
4	Remove Delete exactly the user's composite-key row.

Integrity and implementation notes

- The composite primary key prevents duplicate watches.
 - Watching does not grant permission to bid or join as an authenticated participant.
 - The auction index helps cancellation notification fan-out to watchers.

ENGAGEMENT

auction_participants

Purpose

Persists an authenticated user's Live Arena participation state and last-seen time for lobby/history support.

Related requirements: LIV-001, LIV-002

Relationships

- A user can participate in many auctions; one auction can have many participants.

Field reference

Field	Type	Rules	Meaning
auction_id	uuid	Composite PK, FK auctions.id	Live Arena auction.
user_id	uuid	Composite PK, FK users.id	Authenticated participant.
status	participant_status	Required, JOINED default	RSVP, JOINED, or LEFT.
joined_at	timestamptz	Required	First/current join time.
last_seen_at	timestamptz	Optional	Most recent heartbeat or presence update.

Use case

User C joins the pre-auction lobby. The row becomes JOINED and last_seen_at is refreshed. A timeout marks the participant LEFT after the WebSocket disappears.

Table workflow

1	RSVP or join Upsert the auction-user pair with the intended status.
2	Connect Join the WebSocket room and set JOINED.
3	Heartbeat Refresh last_seen_at at a bounded interval if persistent presence is required.
4	Leave/timeout Set LEFT on explicit leave or stale heartbeat.

Integrity and implementation notes

- The immediate live count should primarily use active WebSocket room membership, not stale database rows.
 - RSVP is optional product behavior and distinct from watchlists.
 - The composite primary key keeps one current participation record per user and auction.

ENGAGEMENT

notifications

Purpose

Stores in-app notification records for outbid, winner, seller-result, and cancellation events.

Related requirements: NOT-001 through NOT-004

Relationships

- Many notifications belong to one user.
 - auction_id and bid_id provide optional context and navigation targets.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Notification identifier.
user_id	uuid	Required, FK users.id	Recipient.
auction_id	uuid	Optional, FK auctions.id	Related auction.
bid_id	uuid	Optional, FK bids.id	Related accepted bid.
type	notification_type	Required	OUTBID, AUCTION_WON, AUCTION_ENDED, or AUCTION_CANCELLED.
title	varchar(180)	Required	Short human-readable heading.
message	varchar(800)	Required	Human-readable explanation.
payload	jsonb	Optional	Structured client navigation/display metadata.
read_at	timestampz	Optional	Null until the user marks it read.
created_at	timestampz	Required	Notification creation time.

Use case

Bidder C takes the lead. The previous leader receives OUTBID linked to the auction and winning bid context, and an authenticated WebSocket event announces the new notification.

Table workflow

1	Trigger Committed bid, completion, or cancellation logic determines recipients.
2	Persist Insert notification rows with safe buyer-facing content.
3	Deliver Emit notification:created to the recipient's authenticated socket.
4	Read Notification list queries unread-first; marking read sets read_at.

Integrity and implementation notes

- V1 is in-app only; email, SMS, and push delivery are deferred.
 - Do not put reserve amount or private bidder identity in message or payload.
 - The (user_id, read_at, created_at) index supports notification-center queries.

Governance

2 tables

Domain responsibility

User reports and append-only administrator actions for accountable moderation.

GOVERNANCE

auction_reports

Purpose

Tracks user-submitted trust-and-safety reports and the administrator's review outcome.

Related requirements: REP-001, REP-002

Relationships

- Each report targets one auction and is submitted by one user.
- reviewed_by_admin_id identifies the administrator who handled it.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Report identifier.
auction_id	uuid	Required, FK auctions.id	Reported auction.
reporter_id	uuid	Required, FK users.id	Authenticated reporter.
reason	report_reason	Required	PROHIBITED_ITEM, MISLEADING_CONTENT, COPYRIGHT, FRAUD, or OTHER.
details	varchar(1000)	Optional	Reporter-provided context; should be required for OTHER in service validation.
status	report_status	Required, OPEN default	OPEN, REVIEWING, RESOLVED, or DISMISSED.
reviewed_by_admin_id	uuid	Optional, FK users.id	Administrator responsible for the final/current review.
reviewed_at	timestampz	Optional	Review/resolution timestamp.
resolution_note	varchar(1000)	Optional	Administrator explanation.
created_at	timestampz	Required	Submission time.

Use case

A user reports a misleading listing. The report starts OPEN, an admin changes it to REVIEWING, then DISMISSED with a note or RESOLVED with a separate moderation action.

Table workflow

1	Submit Authenticate USER, validate public auction/reason, and create OPEN report.
2	Prevent duplicate Within a transaction, reject or consolidate another unresolved report from the same user for the same auction.
3	Review Admin filters OPEN/REVIEWING queue and records reviewer state.
4	Resolve or dismiss Store final status, reviewer, time, and note.
5	Moderate If action is required, cancel/suspend/update in a separate explicit operation and append admin_actions.

Integrity and implementation notes

- The DBML does not enforce unresolved-only uniqueness; the service transaction must do so, or a custom partial unique index may be added.
- RESOLVED means action/decision completed; DISMISSED means no actionable violation.
- Reports are moderation records, not analytics reports.

GOVERNANCE

admin_actions

Purpose

Append-only audit trail for security-sensitive administrator operations affecting users, auctions, categories, and reports.

Related requirements: ADM-001 through ADM-004, REP-002

Relationships

- admin_user_id always references the acting administrator.
- Optional target_user_id, auction_id, and category_id identify the affected record.

Field reference

Field	Type	Rules	Meaning
id	uuid	PK, generated	Audit action identifier.
admin_user_id	uuid	Required, FK users.id	Administrator who performed the action.
target_user_id	uuid	Optional, FK users.id	Affected user for suspension/reactivation.
auction_id	uuid	Optional, FK auctions.id	Affected auction for cancellation/report handling.
category_id	uuid	Optional, FK categories.id	Affected category.
action_type	admin_action_type	Required	SUSPEND_USER, REACTIVATE_USER, CANCEL_AUCTION, CREATE_CATEGORY, UPDATE_CATEGORY, DEACTIVATE_CATEGORY, or RESOLVE_REPORT.
note	varchar(1000)	Optional	Human-readable reason or decision context.
created_at	timestampz	Required	Action time.

Use case

Admin A cancels Auction 123 for a prohibited item. The auction becomes CANCELLED and a CANCEL_AUCTION audit row records admin, auction, note, and time in the same transaction.

Table workflow

1	Authorize Require ADMIN role and ACTIVE status.
2	Validate target Check action-specific target fields and allowed state transition.
3	Mutate Update the user, auction, category, or report.
4	Audit Insert admin_actions in the same transaction.
5	Observe Admin history queries filter by admin, action type, and time.

Integrity and implementation notes

- Rows should not be edited or deleted through normal application APIs.
- Auction reinstatement and destructive category deletion are intentionally absent from V1.
- Validate that the optional target matching action_type is present; the schema alone cannot express that conditional rule.

End-to-end workflows

Table-level correctness is not enough for a real-time auction platform. These workflows show the transaction and event boundaries across multiple tables.

Transaction rule

Persist authoritative state, audit/event rows, and required notifications in a database transaction where consistency matters. Broadcast WebSocket events only after commit.

Local registration and session

Requirements: AUTH-001, AUTH-002, AUTH-004

Tables involved

Primary tables: users, user_profiles, user_sessions

1	Register Insert users and user_profiles atomically after validating names, display name, email, and password.
2	Hash secret Store a password hash, never a password.
3	Login Require ACTIVE status and valid password, then update last_login_at.
4	Create session Insert user_sessions with a unique refresh-token hash and expiry; return short access token and secure refresh cookie.
5	Refresh/logout Rotate a valid refresh token or set revoked_at on logout.

Google login

Requirements: AUTH-003, AUTH-004

Tables involved

Primary tables: auth_accounts, users, user_profiles, user_sessions

1	OAuth callback Validate Google identity and obtain stable provider_account_id.
2	Resolve link Find auth_accounts by GOOGLE plus provider id.
3	Provision If new, create users, user_profiles, and auth_accounts in one transaction.
4	Session Create user_sessions exactly as local login does.

Create, preview, and publish auction

Requirements: AUC-001 through AUC-006

Tables involved

Primary tables: auctions, auction_images, categories, auction_events

1	Create draft Insert auctions in DRAFT with seller, category, pricing, and schedule inputs.
2	Add images Insert ordered auction_images metadata.
3	Preview Render buyer-facing output without changing status.
4	Validate Require active category, image, positive pricing, reserve rule, and valid times.
5	Publish Set SCHEDULED for future start or ACTIVE when start is due; append PUBLISHED event.

Scheduled activation and Live Arena

Requirements: AUC-005, LIV-001, LIV-002

Tables involved

Primary tables: auctions, watchlists, auction_participants, auction_events

1	Discover Guests/users query SCHEDULED auctions by scheduled_start_at.
2	Watch/join User inserts watchlists and joins auction_participants/WebSocket room.
3	Activate Lifecycle worker atomically sets ACTIVE and started_at and appends STARTED.
4	Broadcast Gateway emits auction:started so lobby clients transition without refresh.

Atomic bid, outbid, and anti-sniping

Requirements: BID-001 through BID-004, NOT-001, LIV-003

Tables involved

Primary tables: bids, auctions, auction_extensions, auction_events, notifications

1	Receive idempotent request Use client_request_id to return the previous result for retries.
2	Serialize auction Lock or otherwise protect the current auction state.
3	Validate bid Check ACTIVE status, user role/status, seller restriction, deadline, and minimum amount.
4	Persist atomically Insert bids and update auctions current_price, bid_count, row_version, and reserve state.
5	Extend if needed Insert auction_extensions and update current_end_at/count when inside the final two minutes and below cap.
6	Record side effects Append BID_PLACED/EXTENDED events and OUTBID notification for the prior leader.
7	Broadcast after commit Emit auction:bid, auction:extension, and notification:created.

Auction completion

Requirements: AUC-007, NOT-002, NOT-003, LIV-004

Tables involved

Primary tables: auctions, bids, auction_events, notifications

1	Find due auction Lifecycle worker queries ACTIVE rows by current_end_at.
2	Determine result No valid bids or reserve not met means UNSOLD; otherwise SOLD.
3	Persist terminal state Set ended_at and, for SOLD, winner_user_id, winning_bid_id, and sold_price.
4	Append event Insert ENDED with safe result payload.
5	Notify Create AUCTION_WON for winner and AUCTION_ENDED for seller.
6	Broadcast Emit auction:ended and render the result state.

Report and administrator moderation

Requirements: REP-001, REP-002, ADM-001, ADM-004

Tables involved

Primary tables: auction_reports, admin_actions, auctions, notifications, auction_events

1	Report Authenticated USER creates OPEN auction_reports after duplicate-unresolved check.
2	Review ADMIN moves report to REVIEWING and examines listing/context.
3	Decide Set RESOLVED or DISMISSED with reviewer, time, and note.
4	Act If required, cancel auction and append CANCEL_AUCTION and RESOLVE_REPORT audit actions.
5	Notify Create cancellation notifications for affected users when appropriate.

Suspend or reactivate user

Requirements: ADM-002, ADM-004

Tables involved

Primary tables: users, user_sessions, admin_actions

1	Authorize admin Require ACTIVE ADMIN.
2	Suspend Set users.status = SUSPENDED and revoke all unrevoked user_sessions in one transaction.
3	Audit Insert SUSPEND_USER with target_user_id and note.
4	Enforce Login, refresh, auction creation, and bidding reject the suspended account.
5	Reactivate Set ACTIVE and append REACTIVATE_USER; old revoked sessions stay revoked.

Transaction boundaries and invariants

Operation	Atomic records	Invariant
Register	users + user_profiles (+ auth_accounts for Google)	Avoid an account without its required profile.
Refresh rotation	user_sessions	Replace token hash only after validating session, expiry, revocation, and user status.
Publish	auctions + auction_events	Persist valid status/schedule and its PUBLISHED history together.
Accept bid	bids + auctions + optional auction_extensions + auction_events + notifications	Price, count, reserve, sequence, extension, and outbid state must agree.
Complete auction	auctions + auction_events + notifications	Terminal result, winner/seller alerts, and event must describe the same outcome.
Suspend user	users + user_sessions + admin_actions	Account status and session revocation must not diverge.
Moderate auction	auctions + auction_reports + admin_actions + auction_events	Resolution, cancellation, and audit must remain traceable.
Deactivate category	categories + admin_actions	Category availability and audit reason change together.

Cross-table invariants

- ADMIN accounts never own marketplace auctions or accepted bids.
 - seller_id never equals bidder_id for an accepted bid on the same auction.
- SOLD requires winner_user_id, winning_bid_id, sold_price, and ended_at.
- UNSOLD requires no winner/winning bid and covers no bids or reserve not met.
- current_end_at is at least original_end_at and extension_count is at most five.
- An accepted bid amount is at least the prior current price plus min_bid_increment.
- Only active categories can be selected for publication, but inactive categories remain attached to historical auctions.
- Reserve price and private bidder identity never leave authorized server paths.

Recommended implementation sequence

1	Prisma foundation Translate the approved V1 DBML into schema.prisma, including the (status, bid_count) Hot Auctions index; validate and migrate the initial database.
2	Identity Implement users, user_profiles, auth_accounts, and user_sessions with role/status guards.
3	Taxonomy Implement category read/admin operations and append administrator audit actions.
4	Auction management Implement DRAFT through publish, images, lifecycle fields, and public Starting Soon, Ending Soon, Recently Published, and Hot Auctions queries.
5	Bidding engine Build the atomic bid use case, idempotency, accepted bid_count maintenance, derived reserveMet state, and concurrency tests before WebSocket polish.
6	Lifecycle workers Implement scheduled activation and terminal SOLD/UNSOLD completion.
7	Engagement Add watchlists, persistent notifications, and gateway delivery.
8	Governance Add report queue, resolution, user suspension, auction cancellation, and audit views.
9	E2E acceptance Run the full two-client scheduled auction journey from a clean Docker database, including Hot Auctions order after accepted bids.

Scope guard

Do not add is_featured, is_hot, popularity-score storage, live-viewer ranking, or reserve_met_at to the V1 migration.
Keep Hot Auctions as an Active-auction bid_count query.

Appendix: V1 scope boundaries

Included

- Local and Google authentication with JWT refresh sessions
- User profiles and administrator-managed categories
- Scheduled auctions, images, hidden reserve, and required bid increment
- Starting Soon, Ending Soon, Recently Published, and simple bid-count Hot Auctions
- Atomic real-time bidding, anti-sniping cap, and derived reserveMet broadcast
- Watchlist, in-app notifications, and bounded Live Arena
- Auction reports, user suspension/reactivation, category deactivation, and audit actions

Deferred

- Administrator-curated Featured Auctions and is_featured
- Watcher, participant, live-viewer, time-decay, and stored popularity scoring
- Password recovery and external reset email
- Session IP history, geolocation, and device/user-agent metadata
- Payments, orders, shipping, refunds, disputes, and payouts
- Messaging, reviews, storefronts, seller verification, and advanced analytics
- Email, SMS, push delivery, Redis, horizontal WebSocket scaling, PWA, and native apps

Project references

- docs/requirements/SRS-v1.1.md
- docs/requirements/REQUIREMENTS_TRACEABILITY.md
- docs/architecture/adr/0001-core-domain-decisions.md
- docs/architecture/erd/v1/cbeave-erd-v1.dbml
- PROJECT_ROADMAP.md

Document status

This guide reflects the approved V1 source documents as amended on 2026-07-15. Prisma migrations become executable truth after the Auction model is implemented.